


```

for (int i = 0; i < 26; i++) {
    int count = freq[i];
    char c = i + 'a';

    for (int j = 0; j < count / 2; j++) {
        pairs[pair_idx++] = c;
    }

    if (count % 2 == 1) {
        centers[center_idx++] = c;
    }
}
pairs[pair_idx] = '\0';
centers[center_idx] = '\0';

int k = center_idx;
int m = pair_idx;

if (k == 0) {
    printf("1\n");
    printf("%s", pairs);
    for (int i = m - 1; i >= 0; i--) {
        printf("%c", pairs[i]);
    }
    printf("\n");
}
else {
    printf("%d\n", k);

    int pairs_per_pali = m / k;
    int extra_pairs = m % k;
    int current_pair_start = 0;

    for (int i = 0; i < k; i++) {
        int num_pairs = pairs_per_pali + (i < extra_pairs
? 1 : 0);

        for (int j = 0; j < num_pairs; j++) {
            printf("%c", pairs[current_pair_start + j]);
        }
    }
}

```

```

        printf("%c", centers[i]);

        for (int j = num_pairs - 1; j >= 0; j--) {
            printf("%c", pairs[current_pair_start + j]);
        }
        printf("\n");

        current_pair_start += num_pairs;
    }
}

return 0;
}

```

Solusi Problem K menggunakan pendekatan greedy. Setiap palindrom hanya boleh memiliki maksimal satu huruf ganjil, sehingga jumlah palindrom minimum sama dengan jumlah huruf yang frekuensinya ganjil. Huruf dengan frekuensi genap dibagikan secara merata ke setiap palindrom, sementara huruf ganjil menjadi pusat palindrom. Pendekatan ini menjamin jumlah palindrom minimum. Kompleksitas waktu algoritma ini adalah $O(n)$, dengan n itu adalah panjang string.

Kodingan E:

```

#include <stdio.h>

struct Module {
    char t;
    int k;
};

struct Module modules[200005];
int N, Q;

int main() {
    scanf("%d %d", &N, &Q);

    for (int i = 1; i <= N; i++) {
        scanf(" %c %d", &modules[i].t, &modules[i].k);
    }
    for (int q = 0; q < Q; q++) {
        int i, k;
        char t;
    }
}

```

```

scanf("%d %c %d", &i, &t, &k);

modules[i].t = t;
modules[i].k = k;

for (int j = 1; j <= N; j++) {
    if (modules[j].t == '+') {
        current_value = (current_value + modules[j].k)
% 16;
    } else {
        current_value = (current_value ^
modules[j].k);
    }
}

printf("%d\n", current_value);
}

return 0;
}

```

Solusi Problem E didasarkan pada segment tree dengan komposisi fungsi. Setiap modul dianggap sebagai fungsi dari nilai 0–15, dan rangkaian modul merupakan hasil komposisi fungsi-fungsi tersebut. Karena komposisi fungsi bersifat asosiatif, segment tree digunakan agar setiap perubahan modul dapat diperbarui secara efisien. Dengan teknik ini, setiap update diproses dalam $O(\log N)$ dan keseluruhan kompleksitas menjadi $O((N + Q) \log N)$, jauh lebih efisien dibandingkan perhitungan ulang secara langsung

3.

Masalah F merupakan masalah yang dipilih untuk diselesaikan. Pada masalah F meminta mencari sekumpulan bilangan bulat dalam rentang $[L, R]$ yang jika dirata-rata hasilnya adalah X . Lalu menjumlahkan bilangan sesedikit mungkin. Untuk mengerjakan solusi ini, menggunakan algoritma greedy dan menggunakan pendekatan matematika untuk mengubah desimal ke dalam bentuk pecahan.

Rata - rata merupakan total skor/ jumlah orang . Sehingga jika kita misalnya ingin mengubah 3.5 kedalam bentuk pecahan. Jika diubah akan menjadi $2/7$. Validasi terhadap rata - rata X yang dimana X lebih kecil dari L atau dibanding dari R . Maka rangkaian angka tersebut tidak memungkinkan.

Melakukan konstruksi greedy dengan cara misalnya jika ada 2 orang yang memiliki total skor 7 dari rentang 1 sampai 5. Dengan cara memberi nilai minimal kepada orang tersebut. Menghitung sisa poin untuk mencapai target total. Membagikan sisa poin ke orang pertama sampai maksimal lalu orang kedua dan seterusnya.

Algoritma yang pertama kali dilakukan yaitu melakukan pembacaan input x sebagai string untuk melihat posisi desimal. Mengubah string menjadi pecahan p/q . Jika tidak ada titik, penyebutnya 1. Jika ada, penyebutnya 10^k sesuai jumlah angka di belakang koma. Proses dimulai dengan menghitung Faktor Persekutuan Terbesar (FPB) dari pembilang dan penyebut, lalu kedua nilai tersebut dibagi dengan FPB agar diperoleh bentuk pecahan paling sederhana. Setelah disederhanakan, penyebut dianggap sebagai jumlah elemen (k), sedangkan pembilang menjadi Target Total (sum). Selanjutnya dilakukan pengecekan kondisi batas, yaitu memastikan bahwa nilai rata-rata, yang diperoleh dari sum dibagi k , masih berada di dalam rentang L sampai R . Jika nilai sum dibagi k lebih kecil dari L atau lebih besar dari R , maka solusi tidak mungkin dicapai dan program mencetak -1.

Apabila kondisi tersebut terpenuhi, algoritma greedy dijalankan dengan mencetak jumlah elemen k . Setiap elemen awalnya diasumsikan bernilai L , sehingga dihitung SisaTarget sebagai sum dikurangi hasil perkalian L dengan k . Nilai SisaTarget ini menunjukkan total tambahan yang masih perlu dibagikan. Kemudian dilakukan perulangan sebanyak k kali, di mana pada setiap langkah ditentukan jumlah maksimum yang dapat ditambahkan ke satu elemen, yaitu sebesar R dikurangi L . Dari nilai tersebut, diambil nilai terkecil antara SisaTarget dan R dikurangi L , lalu hasilnya ditambahkan ke L dan dicetak sebagai nilai elemen. Setelah itu, SisaTarget dikurangi sesuai nilai yang sudah ditambahkan, hingga seluruh Target Total berhasil dipenuhi.

Implementasi Kode (Bahasa C)

```

#include <stdio.h>
#include <string.h>

Long Long fpb(Long Long a, Long Long b) {
    if (b == 0) return a;
    return fpb(b, a % b);
}

int main() {
    char nilaiX[100];
    Long Long batasBawah, batasAtas;

    scanf("%s", nilaiX);
    scanf("%lld %lld", &batasBawah, &batasAtas);

    Long Long pembilang = 0, penyebut = 1;
    int titik = -1;
    int panjang = strlen(nilaiX);

    for (int i = 0; i < panjang; i++) {
        if (nilaiX[i] == '.') {
            titik = i;
            break;
        }
    }

    if (titik == -1) {
        for (int i = 0; i < panjang; i++) {
            pembilang = pembilang * 10 + (nilaiX[i] - '0');
        }
    } else {
        for (int i = 0; i < panjang; i++) {
            if (nilaiX[i] != '.') {
                pembilang = pembilang * 10 + (nilaiX[i] - '0');
            }
        }
        for (int i = 0; i < panjang - titik - 1; i++) {
            penyebut *= 10;
        }
    }

    Long Long faktor = fpb(pembilang, penyebut);
    pembilang /= faktor;
    penyebut /= faktor;

    Long Long k = penyebut;
    Long Long totalSkor = pembilang;

    if (totalSkor < batasBawah * k || totalSkor > batasAtas * k) {
        printf("-1\n");
        return 0;
    }

    printf("%lld\n", k);

    Long Long sisaSkor = totalSkor - (batasBawah * k);
    Long Long rangeMaksimal = batasAtas - batasBawah;

    for (int i = 0; i < k; i++) {
        Long Long tambah = (sisaSkor > rangeMaksimal) ? rangeMaksimal : sisaSkor;

        printf("%lld ", batasBawah + tambah);

        sisaSkor -= tambah;
    }
    printf("\n");

    return 0;
}

```

Fungsi Pembantu (FPB)

```
1 long long fpb(long long a, long long b) {  
5     if (b == 0) return a;  
6     return fpb(b, a % b);  
7 }  
8
```

Tujuan dari langkah ini adalah untuk mencari Faktor Persekutuan Terbesar (Greatest Common Divisor). Langkah ini diperlukan karena nilai rata-rata X pada awalnya dapat berupa bilangan desimal, misalnya 3.5, yang kemudian diubah ke dalam bentuk pecahan seperti 35 per 10. Dengan menghitung FPB dari pembilang dan penyebut, pecahan tersebut dapat disederhanakan menjadi bentuk paling sederhana, misalnya 7 per 2, sehingga nilai pembilang dan penyebut menjadi lebih kecil, lebih ringkas, dan lebih mudah digunakan pada proses perhitungan selanjutnya.

Input & Persiapan Variabel

```
1 int main() {  
2     char nilaiX[100];  
3     long long batasBawah, batasAtas;  
4  
5     scanf("%s", nilaiX);  
6     scanf("%lld %lld", &batasBawah, &batasAtas);  
7  
8     long long pembilang = 0, penyebut = 1;  
9     int titik = -1;  
10    int panjang = strlen(nilaiX);  
11 }
```

Nilai X dibaca sebagai *string* karena penggunaan tipe data *float* atau *double* sering menimbulkan masalah presisi pada perhitungan di komputer. Dengan merepresentasikan nilai X sebagai string, setiap digit dapat diolah secara manual dan lebih akurat sesuai kebutuhan algoritma. Selanjutnya, disiapkan variabel pembilang dan penyebut untuk menampung bentuk pecahan dari nilai X, di mana pada tahap awal penyebut diinisialisasi dengan nilai 1 sebagai asumsi bahwa X merupakan bilangan bulat sebelum diproses lebih lanjut.

Mencari Desimal

```
for (int i = 0; i < panjang; i++) {
    if (nilaiX[i] == '.') {
        titik = i;
        break;
    }
}
```

Tujuan dari loop ini adalah untuk mengecek apakah terdapat tanda titik (.) pada input atau tidak, serta menentukan posisinya jika memang ada. Selama proses iterasi, setiap karakter diperiksa satu per satu hingga ditemukan tanda titik, kemudian indeksnya disimpan. Apabila tidak ditemukan tanda titik sama sekali, maka nilai posisi titik tetap bernilai -1, yang menandakan bahwa input tersebut merupakan bilangan bulat tanpa bagian desimal.

Mengubah String ke Angka

Kode memisahkan logika jadi dua kondisi:

Kondisi A: Bilangan Bulat (Gak ada titik)

```
if (titik == -1) {
    for (int i = 0; i < panjang; i++) {
        pembilang = pembilang * 10 + (nilaiX[i] - '0');
    }
}
```

Jika input berupa bilangan bulat, misalnya "5", maka nilai tersebut langsung diubah menjadi angka 5 dengan penyebut tetap bernilai 1, sehingga bentuk pecahannya adalah 5 per 1. Sebaliknya, untuk kondisi bilangan desimal yang memiliki tanda titik, seperti input "3.5", proses dilakukan dengan memisahkan digit-digitnya.

Kondisi B: Bilangan Desimal (Ada titik)

```

} else {
    for (int i = 0; i < panjang; i++) {
        if (nilaiX[i] != '.') {
            pembilang = pembilang * 10 + (nilaiX[i] - '0');
        }
    }
    for (int i = 0; i < panjang - titik - 1; i++) {
        penyebut *= 10;
    }
}

```

Pada loop pertama, angka sebelum dan sesudah titik, yaitu '3' dan '5', digabungkan menjadi 35 dan disimpan sebagai pembilang. Pada loop kedua, dihitung jumlah digit di belakang koma, yang dalam contoh ini berjumlah satu digit, sehingga penyebut dikalikan 10 satu kali dan menjadi 10. Dengan demikian, diperoleh hasil sementara dalam bentuk pecahan 35 per 10.

Penyederhanaan Pecahan

```

long long faktor = fpb(pembilang, penyebut);
pembilang /= faktor;
penyebut /= faktor;

long long k = penyebut;
long long totalSkor = pembilang;

```

Pada tahap simplifikasi, pecahan 35 per 10 disederhanakan dengan membagi pembilang dan penyebut menggunakan Faktor Persekutuan Terbesar, yaitu 5, sehingga diperoleh pecahan 7 per 2. Hasil ini memiliki makna penting dalam konteks algoritma, di mana nilai k diambil dari penyebut, yaitu 2, yang menunjukkan bahwa dibutuhkan minimal dua bilangan. Sementara itu, pembilang bernilai 7 digunakan sebagai totalSkor, yang berarti jumlah dari kedua bilangan tersebut harus sama dengan 7. Dengan demikian, rata-rata yang dihasilkan tetap bernilai 7 dibagi 2, yaitu 3.5, sehingga sesuai dengan target yang diinginkan.

Cek Validasi

```

if (totalSkor < batasBawah * k || totalSkor > batasAtas * k) {
    printf("-1\n");
    return 0;
}

```

Diketahui target total bernilai 7 dengan jumlah bilangan sebanyak 2 dan rentang nilai yang diperbolehkan adalah 1 sampai 5. Total minimum yang mungkin dibentuk adalah hasil perkalian nilai minimum dengan jumlah bilangan,

yaitu 1 dikali 2 sehingga diperoleh 2, sedangkan total maksimum yang mungkin adalah 5 dikali 2 sehingga diperoleh 10. Karena nilai target 7 berada di antara batas minimum 2 dan maksimum 10, maka solusi dapat dibuat. Sebaliknya, apabila nilai target berada di luar rentang tersebut, maka tidak ada kombinasi bilangan yang memenuhi syarat dan program akan mencetak -1.

Konstruksi Greedy

```
printf("%lld\n", k);

long long sisaSkor = totalSkor - (batasBawah * k);
long long rangeMaksimal = batasAtas - batasBawah;

for (int i = 0; i < k; i++) {
    long long tambah = (sisaSkor > rangeMaksimal) ? rangeMaksimal : sisaSkor;

    printf("%lld ", batasBawah + tambah);

    sisaSkor -= tambah;
}
printf("\n");

return 0;
```

Logika yang digunakan dapat dipahami dengan membayangkan bahwa seluruh elemen pada awalnya diisi dengan nilai batas bawah sebagai nilai minimum. Setelah itu dihitung sisaSkor, yaitu selisih antara target total dan jumlah nilai awal seluruh elemen, yang menunjukkan berapa banyak poin tambahan yang masih perlu dibagikan. Sisa skor ini kemudian didistribusikan menggunakan pendekatan greedy dengan membaginya ke elemen satu per satu mulai dari elemen pertama. Setiap elemen hanya boleh ditambah hingga batas tertentu, yaitu rangeMaksimal, agar nilainya tidak melebihi batas atas.

Sebagai ilustrasi, misalkan target total adalah 7, jumlah elemen yang dibutuhkan 2, dan rentang nilai yang diperbolehkan adalah 1 sampai 5. Awalnya kedua elemen dianggap bernilai 1, sehingga total awal adalah 2 dan sisaSkor menjadi 7 dikurangi 2, yaitu 5. Nilai maksimum yang dapat ditambahkan ke satu elemen adalah 4. Pada elemen pertama, sisaSkor sebesar 5 ingin ditambahkan, tetapi karena batas maksimal hanya 4, maka elemen pertama ditambah 4 dan menjadi 5, sementara sisaSkor berkurang menjadi 1. Pada elemen kedua, sisaSkor yang tersisa adalah 1 dan masih berada dalam batas penambahan, sehingga seluruhnya ditambahkan dan elemen kedua menjadi 2. Setelah sisaSkor habis, proses berhenti dan diperoleh output 5 dan 2, yang jika dirata-ratakan menghasilkan nilai 3.5 sesuai dengan target